UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación, Sede Sololá.
Ingeniero Wilmer Orlando Xeron
Base de Datos 1 I Sec. “A”



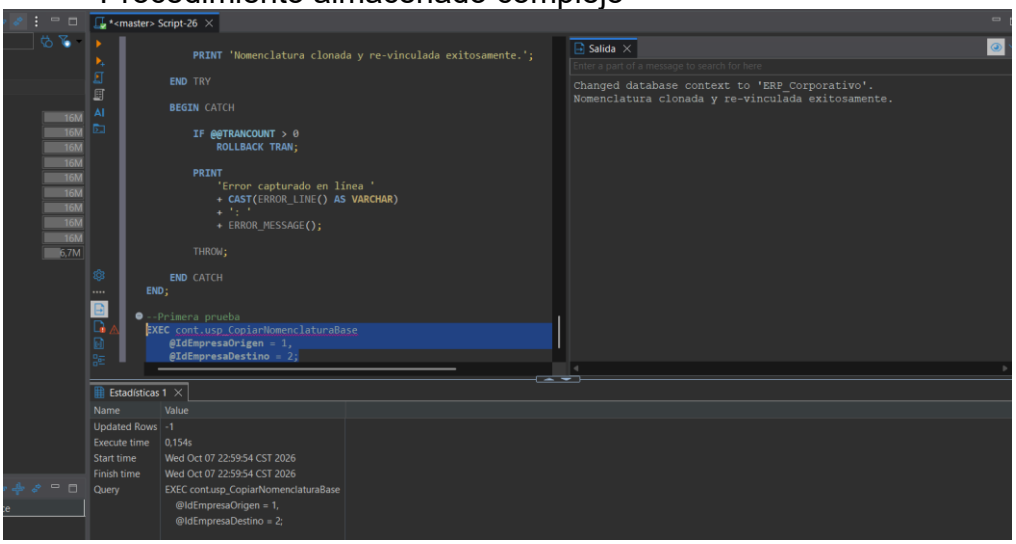
Fase 3: PROGRAMABILIDAD, ACID Y AUDITORÍA DE ALTO NIVEL

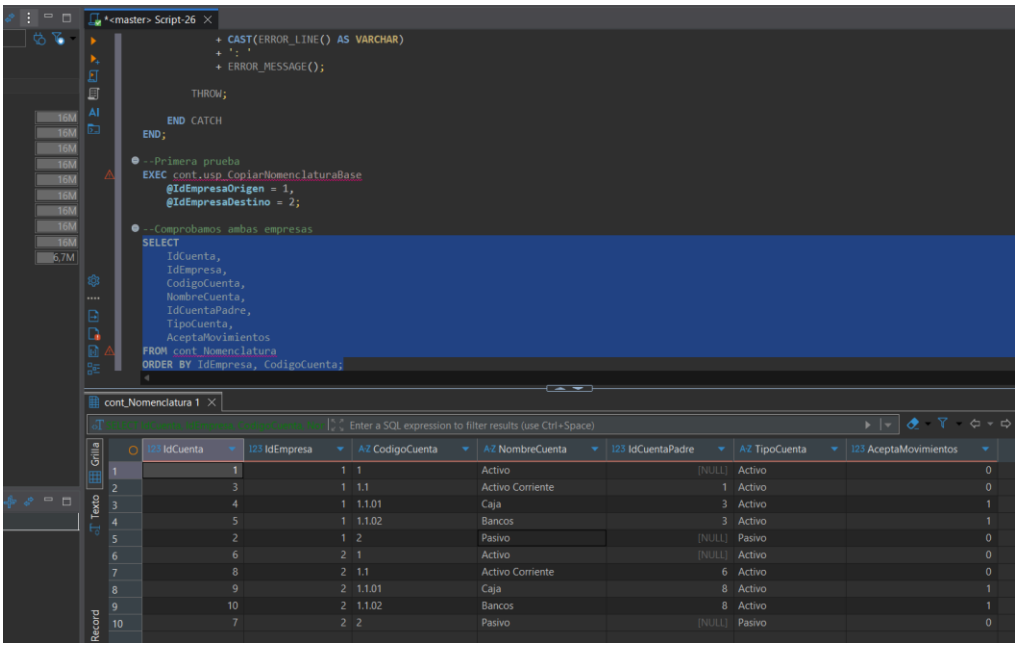
Cristhian Cesar Felipe Mucún Queché
2290-17-17851

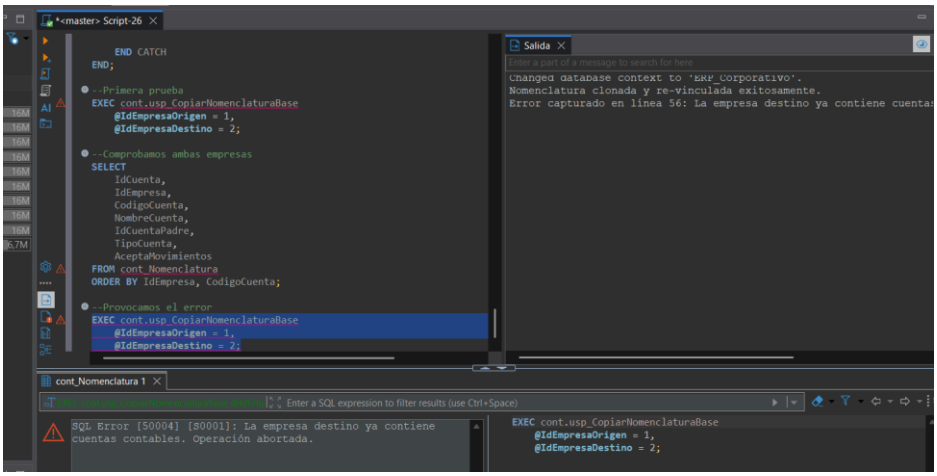
Panajachel 30 de septiembre de 2026

Análisis critico empresarial:
Programabilidad, ACID, y Auditoría de alto nivel

- Procedimiento almacenado complejo

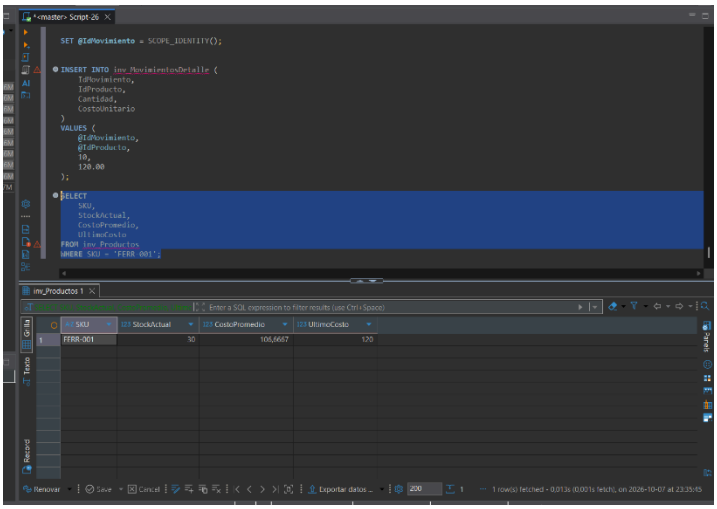


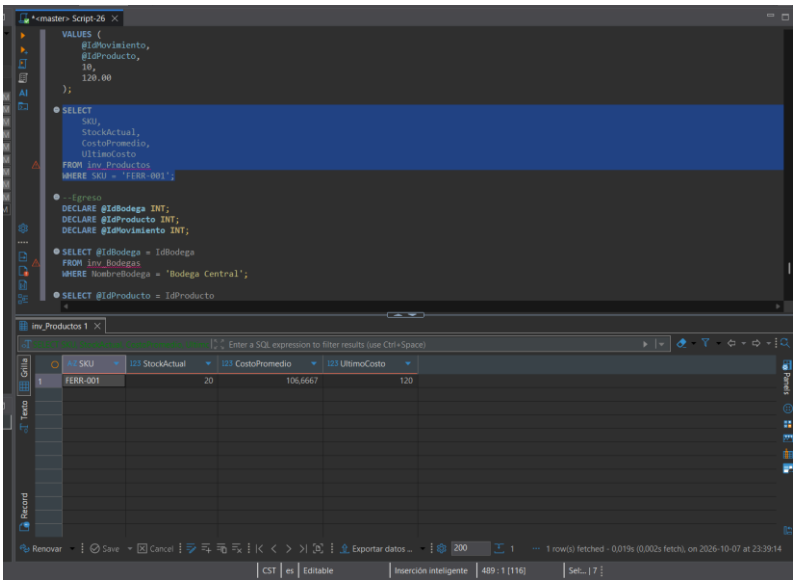




Este procedimiento almacenado automatiza la clonación de una nomenclatura contable entre empresas del ERP. Debido a que las cuentas utilizan una relación recursiva mediante IdCuentaPadre, no es posible copiar directamente los identificadores de la empresa origen, ya que los nuevos registros reciben IDs diferentes. Por ello, la operación primero realiza una clonación plana colocando temporalmente los padres en NULL y posteriormente reconstruye la jerarquía utilizando CodigoCuenta como referencia lógica entre las cuentas originales y sus equivalentes nuevas. Todo el proceso se ejecuta dentro de una transacción con TRY/CATCH, COMMIT y ROLLBACK, protegiendo principalmente la atomicidad y consistencia: si alguna parte falla, no queda una nomenclatura parcialmente copiada, durante el análisis se detectó que el procedimiento depende de que CodigoCuenta identifique de forma única una cuenta dentro de cada empresa, aunque la tabla original no garantizaba esa condición. Se agregó una restricción UNIQUE (IdEmpresa, CodigoCuenta) para eliminar ambigüedades durante la reconstrucción jerárquica.

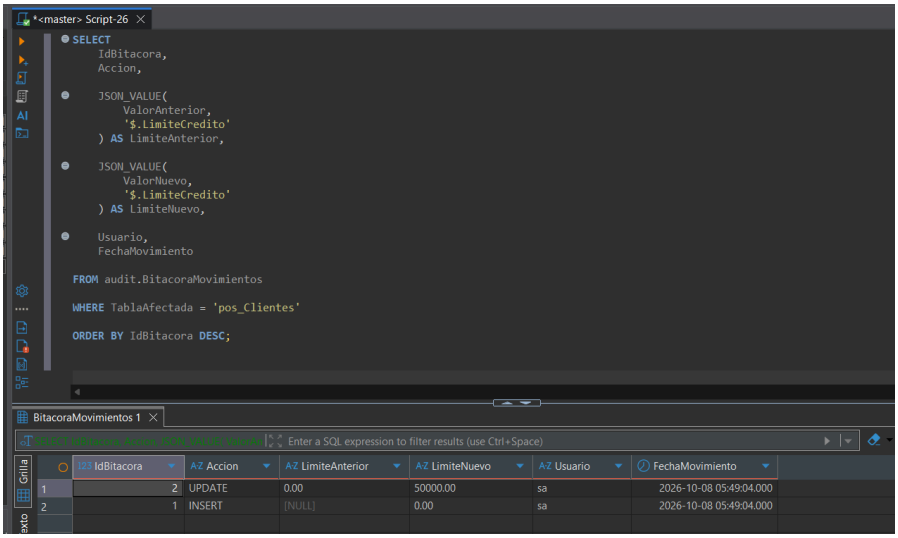
- TRIGGER OPERATIVO: COSTEO PROMEDIO PONDERADO E INVENTARIO





Este trigger automatiza la actualización del inventario cada vez que se insertan detalles de movimientos. Para los ingresos, agrupa las cantidades por producto y recalcula el costo promedio ponderado utilizando el valor del stock existente y el costo de la nueva mercadería; además aumenta las existencias y registra el costo de compra más reciente. Para los egresos se implementó la lógica inversa solicitada en el ejercicio, disminuyendo StockActual sin modificar el costo promedio unitario. El trigger trabaja con el conjunto completo de filas almacenado en INSERTED, por lo que puede procesar múltiples productos dentro de una misma operación sin depender de cursores.

- Auditoría de cliente en JSON



Implementamos una auditoría automática sobre la tabla de clientes mediante un trigger AFTER INSERT, UPDATE, DELETE. SQL Server utiliza las tablas virtuales INSERTED y DELETED para determinar el tipo de operación y obtener las versiones

nueva y anterior del registro. Estos estados se serializan en formato JSON y se almacenan en una bitácora central junto con la tabla afectada, acción realizada, identificador del registro, usuario de SQL Server y fecha del movimiento. Al colocar la auditoría dentro del motor, el control se ejecuta independientemente de si la modificación proviene de DBeaver, Streamlit u otra aplicación.

La bitácora aporta trazabilidad sobre modificaciones sensibles al conservar quién realizó la operación, cuándo ocurrió y cuáles eran los valores antes y después del cambio. A diferencia de una auditoría implementada solamente en el frontend, el trigger centraliza el control dentro de SQL Server y reduce la posibilidad de que una modificación directa a la base quede sin registro.

```
-- Fase 3: Programabilidad, ACID y Auditoría
-- Alumno: Cristhian Mucun 2290-17-17851
USE ERP_Corporativo;


--Creamos empresa destino
INSERT INTO glb_Empresas(
	NIT,
	RazonSocial,
	IdMonedaLocal
)
VALUES (
	'9876543-2',
	'ERP Corporativo Segunda Empresa',
	1
);
--Verificamos que exista
SELECT
	IdEmpresa,
	NIT,
	RazonSocial
```

```
FROM glb_Empresas;

--Creamos una nomenclatura de prueba
INSERT INTO cont_Nomenclatura (
    IdEmpresa,
    CodigoCuenta,
    NombreCuenta,
    IdCuentaPadre,
    TipoCuenta,
    AceptaMovimientos
)
VALUES
(1, '1', 'Activo', NULL, 'Activo', 0),
(1, '2', 'Pasivo', NULL, 'Pasivo', 0);

SELECT --Verificamos que Id's le asignamos
    IdCuenta,
    CodigoCuenta,
    NombreCuenta
FROM cont_Nomenclatura
WHERE IdEmpresa = 1;

-- Creamos la jerarquía
DECLARE @IdActivo INT;
DECLARE @IdActivoCorriente INT;

SELECT @IdActivo = IdCuenta
FROM cont_Nomenclatura
```

```
WHERE IdEmpresa = 1
  AND CodigoCuenta = '1';

INSERT INTO cont_Nomenclatura (
    IdEmpresa,
    CodigoCuenta,
    NombreCuenta,
    IdCuentaPadre,
    TipoCuenta,
    AceptaMovimientos
)
VALUES (
    1,
    '1.1',
    'Activo Corriente',
    @IdActivo,
    'Activo',
    0
);

SELECT @IdActivoCorriente = IdCuenta
FROM cont_Nomenclatura
WHERE IdEmpresa = 1
  AND CodigoCuenta = '1.1';

INSERT INTO cont_Nomenclatura (
    IdEmpresa,
    CodigoCuenta,
```

```
    NombreCuenta,
    IdCuentaPadre,
    TipoCuenta,
    AceptaMovimientos
)
VALUES
(
    1,
    '1.1.01',
    'Caja',
    @IdActivoCorriente,
    'Activo',
    1
),
(
    1,
    '1.1.02',
    'Bancos',
    @IdActivoCorriente,
    'Activo',
    1
);

-- Ahora el Stored Procedure
CREATE OR ALTER PROCEDURE cont.usp_CopiarNomenclaturaBase
    @IdEmpresaOrigen INT,
    @IdEmpresaDestino INT
AS
```

```
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY

        BEGIN TRAN;

        -- Validar que la empresa origen exista
        IF NOT EXISTS (
            SELECT 1
            FROM glb_Empresas
            WHERE IdEmpresa = @IdEmpresaOrigen
        )
        BEGIN
            THROW 50001,
                'La empresa origen no existe.',
                1;
        END;

        -- Validar que la empresa destino exista
        IF NOT EXISTS (
            SELECT 1
            FROM glb_Empresas
            WHERE IdEmpresa = @IdEmpresaDestino
        )
        BEGIN
            THROW 50002,
```

```
        'La empresa destino no existe.',
        1;
END;

-- Validar que la empresa origen tenga cuentas
IF NOT EXISTS (
    SELECT 1
    FROM cont_Nomenclatura
    WHERE IdEmpresa = @IdEmpresaOrigen
)
BEGIN
    THROW 50003,
        'La empresa origen no contiene una nomenclatura para copiar.',
        1;
END;

-- El destino debe estar vacío
IF EXISTS (
    SELECT 1
    FROM cont_Nomenclatura
    WHERE IdEmpresa = @IdEmpresaDestino
)
BEGIN
    THROW 50004,
        'La empresa destino ya contiene cuentas contables. Operación abortada.',
        1;
END;
```

```
-- Paso 1: clonación plana
INSERT INTO cont_Nomenclatura (
    IdEmpresa,
    CodigoCuenta,
    NombreCuenta,
    TipoCuenta,
    AceptaMovimientos,
    IdCuentaPadre
)
SELECT
    @IdEmpresaDestino,
    CodigoCuenta,
    NombreCuenta,
    TipoCuenta,
    AceptaMovimientos,
    NULL
FROM cont_Nomenclatura
WHERE IdEmpresa = @IdEmpresaOrigen;


-- Paso 2: reconstruir relaciones padre-hijo
UPDATE Nuevo

SET Nuevo.IdCuentaPadre = PadreNuevo.IdCuenta

FROM cont_Nomenclatura Nuevo
```

```
    INNER JOIN cont_Nomenclatura Viejo
        ON Nuevo.CodigoCuenta = Viejo.CodigoCuenta
        AND Viejo.IdEmpresa = @IdEmpresaOrigen

    INNER JOIN cont_Nomenclatura PadreViejo
        ON Viejo.IdCuentaPadre = PadreViejo.IdCuenta

    INNER JOIN cont_Nomenclatura PadreNuevo
        ON PadreViejo.CodigoCuenta = PadreNuevo.CodigoCuenta
        AND PadreNuevo.IdEmpresa = @IdEmpresaDestino

    WHERE Nuevo.IdEmpresa = @IdEmpresaDestino;



    COMMIT TRAN;

    PRINT 'Nomenclatura clonada y re-vinculada exitosamente.';

END TRY

BEGIN CATCH

    IF @@TRANCOUNT > 0
        ROLLBACK TRAN;

    PRINT
        'Error capturado en línea '
        + CAST(ERROR_LINE() AS VARCHAR)
```

```
            + ': '
            + ERROR_MESSAGE();

        THROW;

    END CATCH
END;

--Primera prueba
EXEC cont.usp_CopiarNomenclaturaBase
    @IdEmpresaOrigen = 1,
    @IdEmpresaDestino = 2;

--Comprobamos ambas empresas
SELECT
    IdCuenta,
    IdEmpresa,
    CodigoCuenta,
    NombreCuenta,
    IdCuentaPadre,
    TipoCuenta,
    AceptaMovimientos
FROM cont_Nomenclatura
ORDER BY IdEmpresa, CodigoCuenta;

--Provocamos el error
EXEC cont.usp_CopiarNomenclaturaBase
    @IdEmpresaOrigen = 1,
```

```
   @IdEmpresaDestino = 2;

--TRIGGER Operativo
CREATE OR ALTER TRIGGER dbo.trg_ActualizarCostosYStock
ON dbo.inv_MovimientosDetalle
AFTER INSERT
AS
BEGIN
   SET NOCOUNT ON;

 -- 1. Resumimos todos los movimientos insertados por producto

   DECLARE @Resumen TABLE (
      IdProducto INT PRIMARY KEY,
      CantidadIngreso DECIMAL(18,2) NOT NULL,
      CostoTotalIngreso DECIMAL(38,4) NOT NULL,
      CantidadEgreso DECIMAL(18,2) NOT NULL,
      UltimoCostoIngreso DECIMAL(18,4) NULL
   );

   INSERT INTO @Resumen (
      IdProducto,
      CantidadIngreso,
      CostoTotalIngreso,
      CantidadEgreso
   )
   SELECT
      i.IdProducto,
```

```
    SUM(
        CASE
            WHEN m.TipoMovimiento = 'Ingreso'
                THEN i.Cantidad
            ELSE 0
        END
    ),

    SUM(
        CASE
            WHEN m.TipoMovimiento = 'Ingreso'
                THEN i.Cantidad * i.CostoUnitario
            ELSE 0
        END
    ),

    SUM(
        CASE
            WHEN m.TipoMovimiento = 'Egreso'
                THEN i.Cantidad
            ELSE 0
        END
    )

FROM INSERTED i

INNER JOIN dbo.inv_MovimientosMaster m
```

```
    ON i.IdMovimiento = m.IdMovimiento

GROUP BY i.IdProducto;


-- 2. Obtener realmente el último costo de ingreso

UPDATE r

SET r.UltimoCostoIngreso = u.CostoUnitario

FROM @Resumen r

CROSS APPLY (
    SELECT TOP (1)
        i.CostoUnitario

    FROM INSERTED i

    INNER JOIN dbo.inv_MovimientosMaster m
        ON i.IdMovimiento = m.IdMovimiento

    WHERE i.IdProducto = r.IdProducto
      AND m.TipoMovimiento = 'Ingreso'

    ORDER BY i.IdDetalle DESC
) u;
```

```
-- 3. Validar que ningún egreso deje stock negativo

IF EXISTS (
    SELECT 1

    FROM @Resumen r

    INNER JOIN dbo.inv_Productos p
        ON r.IdProducto = p.IdProducto

    WHERE r.CantidadEgreso >
          (p.StockActual + r.CantidadIngreso)
)
BEGIN

    THROW 50010,
        'Stock insuficiente. El egreso solicitado supera las existencias disponibles.',
        1;

END;

-- 4. Actualizar costo promedio, último costo y stock

UPDATE p

SET

    p.CostoPromedio =
```

```
    CASE

        -- Solo recalculamos cuando hubo ingresos
        WHEN r.CantidadIngreso > 0 THEN

            (
                (p.StockActual * p.CostoPromedio)
                +
                r.CostoTotalIngreso
            )
            /
            (
                p.StockActual + r.CantidadIngreso
            )

        -- Un egreso no modifica el promedio unitario
        ELSE p.CostoPromedio

    END,


p.UltimoCosto =
    CASE

        WHEN r.CantidadIngreso > 0
            THEN r.UltimoCostoIngreso

        ELSE p.UltimoCosto
```

```
            END,


        p.StockActual =
            p.StockActual
            + r.CantidadIngreso
            - r.CantidadEgreso


    FROM dbo.inv_Productos p

    INNER JOIN @Resumen r
        ON p.IdProducto = r.IdProducto;

END;

-- Preparando datos para la prueba
IF NOT EXISTS (
    SELECT 1
    FROM inv_Bodegas
    WHERE NombreBodega = 'Bodega Central'
)
BEGIN

    INSERT INTO inv_Bodegas (
        IdSucursal,
        NombreBodega
```

```
    )
    VALUES (
        1,
        'Bodega Central'
    );

END;

IF NOT EXISTS (
    SELECT 1
    FROM inv_Proveedores
    WHERE NIT = '5555555-5'
)
BEGIN

    INSERT INTO inv_Proveedores (
        NIT,
        RazonSocial,
        DiasCredito
    )
    VALUES (
        '5555555-5',
        'Proveedor de Prueba S.A.',
        30
    );

END;
```

```
--Primer y segundo ingreso
DECLARE @IdBodega INT;
DECLARE @IdProveedor INT;
DECLARE @IdProducto INT;
DECLARE @IdMovimiento INT;

SELECT @IdBodega = IdBodega
FROM inv_Bodegas
WHERE NombreBodega = 'Bodega Central';

SELECT @IdProveedor = IdProveedor
FROM inv_Proveedores
WHERE NIT = '5555555-5';

SELECT @IdProducto = IdProducto
FROM inv_Productos
WHERE SKU = 'FERR-001';


INSERT INTO inv_MovimientosMaster (
    IdBodega,
    IdProveedor,
    TipoMovimiento,
    DocumentoRespaldo
)
VALUES (
    @IdBodega,
    @IdProveedor,
```

```
    'Ingreso',
    'COMPRA-001'
);

SET @IdMovimiento = SCOPE_IDENTITY();



INSERT INTO inv_MovimientosDetalle (
    IdMovimiento,
    IdProducto,
    Cantidad,
    CostoUnitario
)
VALUES (
    @IdMovimiento,
    @IdProducto,
    10,
    120.00
);

SELECT
    SKU,
    StockActual,
    CostoPromedio,
    UltimoCosto
FROM inv_Productos
WHERE SKU = 'FERR-001';
```

```
--Egreso
DECLARE @IdBodega INT;
DECLARE @IdProducto INT;
DECLARE @IdMovimiento INT;

SELECT @IdBodega = IdBodega
FROM inv_Bodegas
WHERE NombreBodega = 'Bodega Central';

SELECT @IdProducto = IdProducto
FROM inv_Productos
WHERE SKU = 'FERR-001';


INSERT INTO inv_MovimientosMaster (
    IdBodega,
    IdProveedor,
    TipoMovimiento,
    DocumentoRespaldo
)
VALUES (
    @IdBodega,
    NULL,
    'Egreso',
    'VENTA-001'
);

SET @IdMovimiento = SCOPE_IDENTITY();
```

```
INSERT INTO inv_MovimientosDetalle (
    IdMovimiento,
    IdProducto,
    Cantidad,
    CostoUnitario
)
VALUES (
    @IdMovimiento,
    @IdProducto,
    10,
    120.00
);

-- Auditoria
IF SCHEMA_ID('audit') IS NULL
BEGIN
    EXEC('CREATE SCHEMA audit');
END;

IF OBJECT_ID('audit.BitacoraMovimientos', 'U') IS NULL
BEGIN

    CREATE TABLE audit.BitacoraMovimientos (
        IdBitacora BIGINT IDENTITY(1,1) PRIMARY KEY,

        TablaAfectada VARCHAR(100) NOT NULL,
```

```
Accion VARCHAR(10) NOT NULL
    CONSTRAINT CK_Bitacora_Accion
    CHECK (Accion IN ('INSERT', 'UPDATE', 'DELETE')),

RegistroID INT NOT NULL,

ValorAnterior NVARCHAR(MAX) NULL,

ValorNuevo NVARCHAR(MAX) NULL,

Usuario SYSNAME NULL,

FechaMovimiento DATETIME2(0) NOT NULL
    CONSTRAINT DF_Bitacora_FechaMovimiento
    DEFAULT SYSDATETIME(),

CONSTRAINT CK_Bitacora_ValorAnteriorJSON
    CHECK (
        ValorAnterior IS NULL
        OR ISJSON(ValorAnterior) = 1
    ),

CONSTRAINT CK_Bitacora_ValorNuevoJSON
    CHECK (
        ValorNuevo IS NULL
        OR ISJSON(ValorNuevo) = 1
    )
```

```
    );

END;

CREATE OR ALTER TRIGGER dbo.trg_AuditoriaClientesJSON
ON dbo.pos_Clientes
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @Accion VARCHAR(10);

    -- Si existen filas tanto en INSERTED como en DELETED,
    -- significa que fue un UPDATE.
    IF EXISTS (SELECT 1 FROM INSERTED)
       AND EXISTS (SELECT 1 FROM DELETED)

        SET @Accion = 'UPDATE';



    -- Si solamente existen filas en INSERTED,
    -- significa que fue un INSERT.
    ELSE IF EXISTS (SELECT 1 FROM INSERTED)

        SET @Accion = 'INSERT';
```

```
-- Si solamente existen filas en DELETED,
-- significa que fue un DELETE.
ELSE

    SET @Accion = 'DELETE';

INSERT INTO audit.BitacoraMovimientos (
    TablaAfectada,
    Accion,
    RegistroID,
    ValorAnterior,
    ValorNuevo,
    Usuario
)

SELECT
    'pos_Clientes',

    @Accion,

    COALESCE(i.IdCliente, d.IdCliente),

    -- Estado anterior:
    -- NULL durante un INSERT.
    CASE
        WHEN d.IdCliente IS NULL THEN NULL
        ELSE (
            SELECT d2.*
```

```
            FROM DELETED d2
            WHERE d2.IdCliente = d.IdCliente
            FOR JSON PATH, WITHOUT_ARRAY_WRAPPER
        )
    END,

    -- Estado nuevo:
    -- NULL durante un DELETE.
    CASE
        WHEN i.IdCliente IS NULL THEN NULL
        ELSE (
            SELECT i2.*
            FROM INSERTED i2
            WHERE i2.IdCliente = i.IdCliente
            FOR JSON PATH, WITHOUT_ARRAY_WRAPPER
        )
    END,

    SUSER_SNAME()

FROM INSERTED i

FULL OUTER JOIN DELETED d
    ON i.IdCliente = d.IdCliente;

END;

DECLARE @IdTipoCliente INT;
```

```
DECLARE @IdClientePrueba INT;


SELECT TOP (1)
    @IdTipoCliente = IdTipoCliente
FROM dbo.pos_TiposCliente
WHERE NombreTipo = 'Mayorista VIP';


IF @IdTipoCliente IS NULL
BEGIN

    INSERT INTO dbo.pos_TiposCliente (
        NombreTipo,
        PorcentajeDescuento
    )
    VALUES (
        'Mayorista VIP',
        10.00
    );

    SET @IdTipoCliente = SCOPE_IDENTITY();

END;


-- Crear cliente de prueba
INSERT INTO dbo.pos_Clientes (
```

```
    IdTipoCliente,
    NIT,
    NombreCompleto,
    LimiteCredito
)
VALUES (
    @IdTipoCliente,
    '12345678',
    'Tecnologías Avanzadas S.A.',
    0
);

SET @IdClientePrueba = SCOPE_IDENTITY();



-- Provocar un UPDATE para capturar
-- el valor anterior y el nuevo.
UPDATE dbo.pos_Clientes

SET LimiteCredito = 50000.00

WHERE IdCliente = @IdClientePrueba;

SELECT
    IdBitacora,
    TablaAfectada,
    Accion,
    RegistroID,
```

```
    ValorAnterior,
    ValorNuevo,
    Usuario,
    FechaMovimiento

FROM audit.BitacoraMovimientos
WHERE RegistroID = @IdClientePrueba
ORDER BY IdBitacora;

SELECT
    IdBitacora,
    Accion,

    JSON_VALUE(
        ValorAnterior,
        '$.LimiteCredito'
    ) AS LimiteAnterior,

    JSON_VALUE(
        ValorNuevo,
        '$.LimiteCredito'
    ) AS LimiteNuevo,
    Usuario,
    FechaMovimiento
FROM audit.BitacoraMovimientos
WHERE TablaAfectada = 'pos_Clientes'
ORDER BY IdBitacora DESC;
```